Real-world (a real approval pause) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `approval_pause_practice.py` : this is the exact `interrupt()` pattern the Build Task needs at its final gate — pause with the draft and its sources, and only continue after a human decides. **If not:** the Build Task's approval step would be the first `interrupt()` you ever wired to a real search-and-draft graph.

Uses the same `practice/` setup as the Basic exercise (Doc08's files, `docs/` and `kb.py`).

Basic Version

Approach 1 — the direct way

```
# approval_pause_practice.py
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command, interrupt

from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)


class ApprovalState(TypedDict):
    task: str
    sources: list
    draft: str
    approved: bool
    answer: str


def draft_node(state):
    sources = retrieve(collection, state["task"])
    pieces = []
```

```
    for source in sources:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = "Context:\n" + context + "\n\nQuestion: " + state["task"]
    return {"sources": sources, "draft": model.invoke(prompt).content}


def approval_node(state):
    files = []
    for source in state["sources"]:
        files.append(source["source_file"])
    decision = interrupt({"draft": state["draft"], "sources": files})
    return {"approved": decision == "approved"}


def finish_node(state):
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- nothing was sent."}


builder = StateGraph(ApprovalState)
builder.add_node("draft_node", draft_node)
builder.add_node("approval_node", approval_node)
builder.add_node("finish_node", finish_node)
builder.add_edge(START, "draft_node")
builder.add_edge("draft_node", "approval_node")
builder.add_edge("approval_node", "finish_node")
builder.add_edge("finish_node", END)
graph = builder.compile(checkpointer=MemorySaver())

config = {"configurable": {"thread_id": "thread-1"}}
task = {"task": "How hot should the water be when brewing?"}
paused = graph.invoke(task, config)
print("Paused, sources:", paused["__interrupt__"][0].value["sources"])

final = graph.invoke(Command(resume="approved"), config)
print("Final answer:", final["answer"])
```

Expected output (the wording varies):

```
Paused, sources: ['Water temperature.txt', 'Pour-over basics.txt',
'French press basics.txt']
Final answer: Water for brewing coffee should be between 195 and 205
degrees Fahrenheit.
```

(Wrapped to fit the page.) The first `invoke()` runs `draft_node`, then stops inside `approval_node` at `interrupt()` and returns the payload under `"__interrupt__"`. `Command(resume="approved")` on the same `thread_id` continues from there (Doc09). This only tests the "approved" path.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Approach 1 — type hints, an explicit prompt, and testing both outcomes

```
# approval_pause_practice.py
from typing import TypedDict

from langchain_openai import ChatOpenAI
from langgraph.checkpoint.memory import MemorySaver
from langgraph.graph import END, START, StateGraph
from langgraph.types import Command, interrupt

from kb import collection
from retriever import retrieve

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)


class ApprovalState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    approved: bool
    answer: str


def draft_node(state: ApprovalState) -> dict:
    sources = retrieve(collection, state["task"])
    pieces: list[str] = []
    for source in sources:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {"sources": sources, "draft": model.invoke(prompt).content}


def source_files(state: ApprovalState) -> list[str]:
    files: list[str] = []
    for source in state["sources"]:
        files.append(source["source_file"])
    return files
```

```
def approval_node(state: ApprovalState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "sources": source_files(state),
        "prompt": "Approve this answer before it's sent? (approved/rejected)",
    })
    return {"approved": decision == "approved"}


def finish_node(state: ApprovalState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    return {"answer": "Not approved -- nothing was sent."}


def build_graph():
    builder = StateGraph(ApprovalState)
    builder.add_node("draft_node", draft_node)
    builder.add_node("approval_node", approval_node)
    builder.add_node("finish_node", finish_node)
    builder.add_edge(START, "draft_node")
    builder.add_edge("draft_node", "approval_node")
    builder.add_edge("approval_node", "finish_node")
    builder.add_edge("finish_node", END)
    return builder.compile(checkpointer=MemorySaver())


def run_with_approval(graph, task: str, decision, thread_id: str) -> dict:
    config = {"configurable": {"thread_id": thread_id}}
    graph.invoke({"task": task}, config)
    return graph.invoke(Command(resume=decision), config)


if __name__ == "__main__":
    graph = build_graph()
    task = "How hot should the water be when brewing?"
    approved = run_with_approval(graph, task, "approved", "thread-yes")
    print("approved run:", approved["answer"])
    rejected = run_with_approval(graph, task, "rejected", "thread-no")
    print("rejected run:", rejected["answer"])
```

Expected output:

```
approved run: Water for brewing coffee should be between 195 and 205
degrees Fahrenheit.
rejected run: Not approved -- nothing was sent.
```

Difference from Basic: full type hints, a `prompt` in the interrupt payload so a real reviewer knows what's being asked, and both outcomes tested. Each run gets its **own** `thread_id` — reusing one would mix two runs' saved state. Still missing: who decided, and when.

Approach 2 — a structured resume payload, recording who and when

Story: a bare `"approved"` says *what* was decided but not *who* or *when* — the first question anyone asks when something goes wrong. **If not:** the Build Task's approval step could never show who signed off.

Add below Approach 1's code (it replaces `ApprovalState` and `approval_node`):

```
# approval_pause_practice.py
from datetime import datetime, timezone


class ApprovalState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    approved: bool
    approved_by: str
    approved_at: str
    answer: str


def approval_node(state: ApprovalState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "sources": source_files(state),
        "prompt": "Approve this answer before it's sent?",
    })
    return {
        "approved": decision["decision"] == "approved",
        "approved_by": decision["by"],
        "approved_at": decision["at"],
    }


graph = build_graph()
decision = {
```

```
    "decision": "approved",
    "by": "jane@example.com",
    "at": datetime.now(timezone.utc).isoformat(),
}
final = run_with_approval(
    graph, "How hot should the water be when brewing?", decision, "audit-1"
)
print(f"Approved by {final['approved_by']} at {final['approved_at']}")
```

Expected output:

```
Approved by jane@example.com at 2026-10-01T10:00:00.000000+00:00
```

The resume value can be any JSON-like value, so a dict carries the decision plus who and when — and the node saves them into state instead of throwing them away.

Approach 3 — a staleness check before trusting an old approval

Story: recording *when* only helps if something checks it. A draft that sat waiting for a week may rest on documents that have since changed. **If not:** a month-old approval would be treated exactly like one made two minutes ago.

The catch: when a paused node resumes, LangGraph runs it again **from its first line** — so `datetime.now()` taken inside `approval_node` would be the *resume* time, and the draft would never look old. The time must be saved earlier, in `draft_node`, where it's stored in state once.

Add below Approach 2's code:

```
# approval_pause_practice.py
from datetime import datetime, timedelta, timezone

MAX_APPROVAL_AGE_HOURS = 168  # 1 week


class ApprovalState(TypedDict):
    task: str
    sources: list[dict]
    draft: str
    drafted_at: str
    approved: bool
    approved_by: str
    approved_at: str
    answer: str
```

```
def draft_node(state: ApprovalState) -> dict:
    sources = retrieve(collection, state["task"])
    pieces: list[str] = []
    for source in sources:
        pieces.append(source["text"])
    context = "\n\n".join(pieces)
    prompt = f"Context:\n{context}\n\nQuestion: {state['task']}"
    return {
        "sources": sources,
        "draft": model.invoke(prompt).content,
        # why: saved here, not in approval_node -- a node re-runs from its
        # first line when it resumes, so a time taken there is resume time
        "drafted_at": datetime.now(timezone.utc).isoformat(),
    }


def is_stale(
    drafted_at: str, max_age_hours: int = MAX_APPROVAL_AGE_HOURS
) -> bool:
    age = datetime.now(timezone.utc) - datetime.fromisoformat(drafted_at)
    return age > timedelta(hours=max_age_hours)


def approval_node(state: ApprovalState) -> dict:
    decision = interrupt({
        "draft": state["draft"],
        "sources": source_files(state),
        "drafted_at": state["drafted_at"],
    })
    if is_stale(state["drafted_at"]):
        return {
            "approved": False,
            "answer": "This draft is too old to approve -- search again.",
        }
    return {
        "approved": decision["decision"] == "approved",
        "approved_by": decision["by"],
        "approved_at": decision["at"],
    }


def finish_node(state: ApprovalState) -> dict:
    if state["approved"]:
        return {"answer": state["draft"]}
    if state.get("answer"):
        return {}
    return {"answer": "Not approved -- nothing was sent."}
```

```
recent = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
old = (datetime.now(timezone.utc) - timedelta(days=30)).isoformat()
print("2 hours old, stale?", is_stale(recent))
print("30 days old, stale?", is_stale(old))
```

Expected output:

```
2 hours old, stale? False
30 days old, stale? True
```

`datetime.fromisoformat()` turns the saved text back into a date, and `timedelta(hours=...)` is Doc02's way of measuring "how long ago". A stale draft is not approved, whatever the reviewer said, and `finish_node` keeps the explanation instead of overwriting it.

Difference from Approach 1, and between Approaches 2/3: Approach 1 tests both outcomes with a bare string. Approach 2 records who and when. Approach 3 acts on the "when" — and saves the time where a resume can't reset it.

Which one should you actually write? Always test both outcomes, like Approach 1 — a graph that does the risky thing even on "rejected" is a real bug you only catch by testing rejection. Add Approach 2's who-and-when as soon as more than one person can approve. Add Approach 3 once paused runs can realistically wait hours or days.